KimchiSpeech: Faster, Lighter and More Controllable TTS with Hierarchical VAE and STT-aided Gaussian Attention

Soochul Park

MODULABS, Seoul, Korea

scpark20@gmail.com

Abstract

In this study, we propose KimchiSpeech, a text-to-speech (TTS) model that employs a hierarchical variational autoencoder (VAE) architecture and uses an attention alignment obtained from a speech-to-text (STT) model. The hierarchical VAE architecture contributes to the generation of high-quality speech and diverse prosody. Because the STT model operates independently of the TTS model, the attention alignment can be estimated more robustly. Moreover, the speaking rate of the generated speech can be flexibly controlled through soft attention based on Gaussian distributions. Furthermore, we propose two model configurations, namely KimchiSpeech-W and KimchiSpeech-S. The former is a lightweight version with only 3.3M parameters at inference time, whereas the latter is a fast version that generates speech at 470× real-time speed on a GPU. The mean opinion score (MOS) results show that the generated speech is of state-of-the-art quality. [1]

Index Terms: text-to-speech, speech synthesis, variational autoencoder, attention

1. Introduction

After the advent of Tacotron [1, 2], end-to-end text-to-speech (TTS) systems based on neural networks have been actively studied. Because Tacotron generates mel-spectrograms sequentially as an autoregressive model, its inference is slow. Several non-autoregressive models, such as FastSpeech [3], have been proposed to address this limitation. Non-autoregressive models can be categorized into regression-based, flow-based, and generative adversarial network (GAN) [4]-based models.

Regression-based models include FastSpeech and its successor, FastSpeech2 [5]. Although TTS is a one-to-many mapping that does not naturally fit a regression-based approach, this issue can be mitigated by using a Transformer [6] with strong modeling capacity. In FastSpeech2, speech quality is improved by incorporating additional information, such as pitch and energy.

Flow-based models based on normalizing flows include Flow-TTS [7] and Glow-TTS [8]. They transform the data distribution into a prior distribution through a sequence of bijective functions. Although they require all network layers to be invertible, they enable the generation of speech with diverse prosody by sampling from the prior distribution.

Variational autoencoder (VAE)-based models include BVAE-TTS [9] and VARA-TTS [10]. Flow-based models must use invertible layers, whereas VAE-based models address this limitation by introducing a variational posterior. Similar to flow-based models, they can also sample from the prior distribution.

Additionally, GAN-based models such as GAN-TTS [11], EATS [12], and FastSpeech2s [5] directly generate waveforms.

[1] Audio samples and code are available at `http://scpark20.github.io/KimchiSpeech`.

KimchiSpeech is a VAE-based model that follows the architecture of very deep VAE (VDVAE) [13], which is a type of hierarchical VAE (Section 2.2). By simplifying this architecture, the inference speed can be improved.

Our contributions are summarized as follows:

- We apply a hierarchical VAE to speech synthesis, enabling the rapid generation of high-quality speech.
- We propose a speech-to-text (STT)-aided Gaussian attention mechanism that operates independently of the TTS model, enabling robust attention alignment and flexible control over speech length.
- We provide a weight-optimized model and a speed-optimized model to support use in several applications.

2. Background

2.1. Variational Inference

A variational autoencoder (VAE) [14] is a generative model consisting of an encoder and a decoder. These represent the variational posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ and the conditional distribution $p_\theta(\mathbf{x} \mid \mathbf{z})$, respectively. The objective function is a lower bound on the log-likelihood of the data $\mathbf{x}$.

$$\log p_\theta(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z})) \tag{1}$$

The right-hand side of (1) is called the evidence lower bound (ELBO) and consists of a reconstruction term and a Kullback–Leibler (KL) divergence term. The reconstruction term encourages accurate reconstruction of the data $\mathbf{x}$ from a latent variable $\mathbf{z}$ sampled from the variational posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$. The KL divergence term encourages the variational posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ to remain close to the prior $p_\theta(\mathbf{z})$.

2.2. Hierarchical VAE

In a standard VAE, the prior is assumed to be a fully factorized Gaussian distribution. This assumption is often too restrictive to adequately represent the data distribution, which can result in low-quality outputs. To address this issue, hierarchical models such as LadderVAE [15], NVAE [16], and VDVAE [13] have been proposed. These models hierarchically factorize the prior $p_\theta(\mathbf{z})$ using the chain rule.

$$p_\theta(\mathbf{z}) = p_\theta(\mathbf{z}_0) \prod_{n=1}^{N} p_\theta(\mathbf{z}_n \mid \mathbf{z}_{<n}) \tag{2}$$

The variational posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ can be factorized in a similar manner:

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = q_\phi(\mathbf{z}_0 \mid \mathbf{x}) \prod_{n=1}^{N} q_\phi(\mathbf{z}_n \mid \mathbf{z}_{<n}, \mathbf{x}) \tag{3}$$

Each factorized distribution is modeled by a corresponding layer in the encoder and decoder.

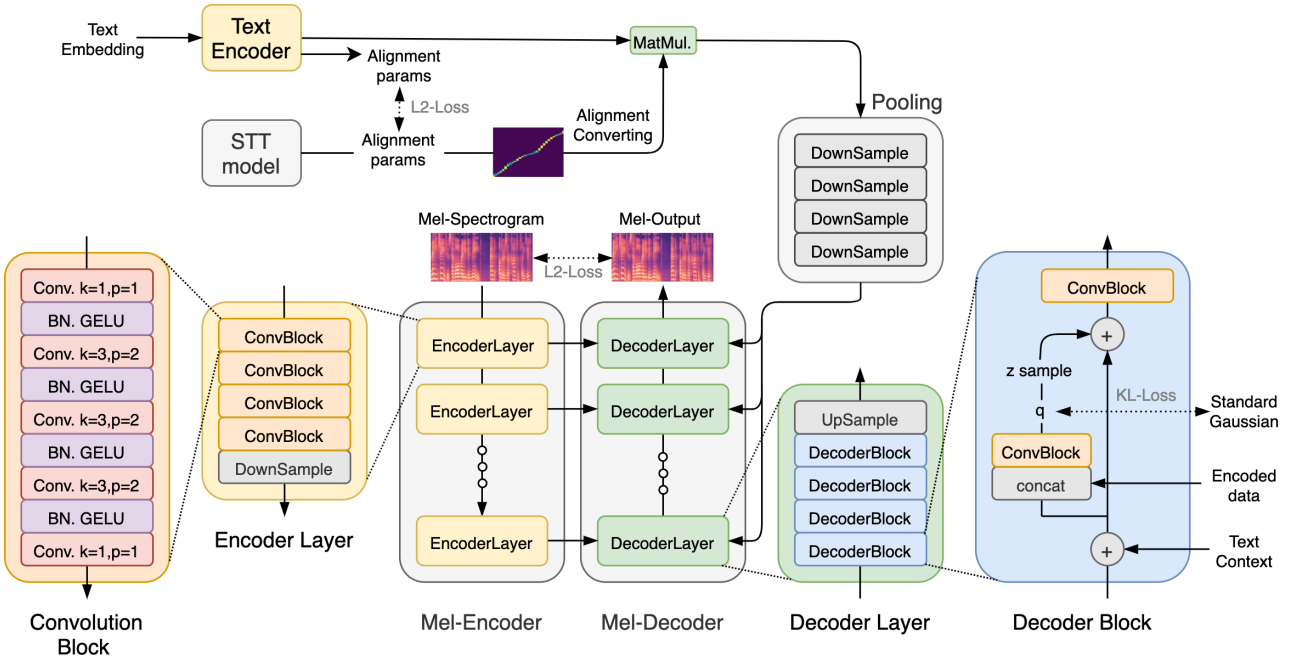

Figure 1: *KimchiSpeech Structure*

3. Model Structure

The proposed model consists of two parts: an STT model and a TTS model. The STT model takes a mel-spectrogram as input and outputs text. However, this model is used only to obtain text-to-mel alignment during training and is not used at inference time. The TTS model takes text as input and generates a mel-spectrogram. In this study, the text-to-mel alignment obtained from the STT model is used to derive a mel-to-text alignment.

3.1. Speech-to-Text

The STT model is structurally similar to Tacotron2 [2]. However, its input and output are reversed, and cross-entropy loss is used as the training objective. The encoder of Tacotron2 functions as a mel encoder, and the decoder functions as a text decoder.

In [17, 18], Gaussian mixture model-based attention was proposed. We simplify this approach to a single-Gaussian attention mechanism and use it in the STT model.

For text index i and mel index j, the attention weight is defined as the value of a Gaussian distribution.

$$\alpha_{ij}^{STT} = \frac{1}{\sigma_i\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{j-\mu_i}{\sigma_i}\right)^2} \tag{4}$$

The intermediate parameters $(\hat{\Delta}_i, \hat{\sigma}_i)$ are obtained by applying a linear transformation to the i-th hidden state s_i of the text decoder. The trainable parameters W and b are initialized to zero.

$$(\hat{\Delta}_i, \hat{\sigma}_i) = W s_i + b \tag{5}$$

The parameters (Δ_i, σ_i) are obtained by applying the exponential function to the intermediate parameters $(\hat{\Delta}_i, \hat{\sigma}_i)$ and multiplying them by appropriate coefficients.

$$\begin{cases} \Delta_i = c_\Delta \exp \hat{\Delta}_i \\ \sigma_i = c_\sigma \exp \hat{\sigma}_i \end{cases} \tag{6}$$

The coefficients c_Δ and c_σ are treated as hyperparameters and are used to produce a proper attention alignment. Empirically, c_Δ should be set close to the average value of $\mathrm{mel;length}/\mathrm{text;length}$. Finally, the mean μ_i is obtained by adding the current offset Δ_i to the mean from the previous step, μ_{i-1}.

$$\begin{cases} \mu_0 = 0 \\ \mu_i = \mu_{i-1} + \Delta_i \end{cases} \tag{7}$$

3.2. Text-to-Speech

The TTS model is built on the VDVAE architecture and one-dimensional convolutions. During training, a condition is constructed from the input text and the attention alignment obtained from the STT model. The input mel-spectrogram is then encoded by the mel encoder. The mel decoder subsequently reconstructs a mel-spectrogram from the condition and the encoded representation.

3.2.1. Text-Encoder

The text encoder is nearly identical to that of Tacotron2. Specifically, it consists of three convolutional layers and one bidirectional LSTM. However, an additional linear layer is introduced to output the attention parameters.

3.2.2. Convolution Block

The convolution block is a fundamental module used to construct the mel encoder and mel decoder. Convolutions with a kernel size of 1 are used in the input and output layers, whereas convolutions with a kernel size of 3 and a padding size of 2 are used in the intermediate layers. Batch normalization [19] and a GELU activation [20] are applied after all convolution layers except the last one.

3.2.3. Mel-Encoder

The mel encoder encodes the input mel-spectrogram, and the resulting representation is used to obtain the parameters of the variational posterior during training. The mel encoder consists of several encoder layers. Furthermore, each mel encoder layer consists of several convolution blocks. After each mel encoder layer, downsampling is performed by a convolution with a kernel size of 2 and a stride of 2, reducing the sequence length by half.

3.2.4. Mel-Decoder

The mel decoder reconstructs the mel-spectrogram from the encoded representation and the given condition. The mel decoder consists of several decoder layers. The number of decoder layers is equal to that of the mel encoder layers. Additionally, each decoder layer consists of several decoder blocks. After each decoder layer, upsampling is performed by a transposed convolution with a kernel size of 2 and a stride of 2, doubling the sequence length.

The decoder block plays a key role in the hierarchical VAE structure. During training, the variational posterior is computed from the encoded representation, the given condition, and the hidden vector output from the lower layer. A latent vector $\mathbf{z}$ is sampled from the variational posterior and added to the hidden vector. The convolution block is then applied, and its output is produced as the final result. At inference time, a latent vector $\mathbf{z}$ is sampled from the prior rather than from the variational posterior.

In this work, we aim to simplify the decoding process for fast inference. Unlike previous studies [10, 13, 16], we set the prior to a standard Gaussian distribution. Therefore, no additional operations are required to obtain the prior. In addition, the sampled $\mathbf{z}$ is simply added to a subvector of the hidden vector. Therefore, no additional operations are required to match the dimension of $\mathbf{z}$ to that of the hidden vector.

3.2.5. STT-aided Gaussian Attention

The text-to-mel alignment obtained from the STT model must be converted into a mel-to-text alignment for use in the TTS model. To this end, we first take the logarithm of each element of the alignment matrix α^{STT} (see Eq. 4).

$$L_{ji}^{TTS} = \log(\alpha_{ij}^{STT} + \epsilon) \tag{8}$$

Here, ϵ is a small constant and is set to 1e-8 in the implementation. We then apply the softmax function along the text axis.

$$\alpha_{ji}^{TTS} = \frac{\exp(L_{ji}^{TTS})}{\sum_{i'} \exp(L_{ji'}^{TTS})} \tag{9}$$

At inference time, a text-to-mel alignment is obtained from the alignment parameters output by the text encoder in the TTS model and is converted into a mel-to-text alignment using the same procedure.

The length of the mel-spectrogram cannot be determined directly from the alignment parameters. To address this issue, an end-of-sequence (EOS) token is appended to the end of the text. Thereafter, in the alignment parameters output by the STT model, the delta value at the EOS token index, Δ_{EOS}, is modified as follows.

$$\Delta_{EOS} = T - \sum_{i}^{L} \Delta_i \tag{10}$$

Here, T is the length of the mel-spectrogram, and L is the length of the text. The text encoder is trained to predict this value. At inference time, the delta values are accumulated, and the mean value at the EOS token index is regarded as the length of the mel-spectrogram.

4. Details

4.1. Training Details

Texts used for training are normalized and converted into phonemes using the grapheme-to-phoneme (G2P) library [21]. This helps ensure correct pronunciation. The Adam optimizer [22], with a learning rate of 1e-4 and a weight decay of 1e-6, was used, and the model was trained for 800k steps with a batch size of 32.

The total loss function $\mathcal{L}_{total}$ consists of the cross-entropy loss $\mathcal{L}_{CE}$ of the STT model, the reconstruction loss $\mathcal{L}_{recon}$, the KL-divergence loss $\mathcal{L}_{KL}$, and the alignment parameter loss $\mathcal{L}_{align}$ of the TTS model.

$$\mathcal{L}_{total} = \mathcal{L}_{recon} + \beta\mathcal{L}_{KL} + \mathcal{L}_{CE} + \mathcal{L}_{align} \tag{11}$$

To prevent posterior collapse, the coefficient of the KL-divergence loss is increased linearly from 0.0 to 1.0 over 50k steps. Although we also experimented with other annealing methods [23], there was no significant difference when training was performed for a sufficiently long time.

4.2. Inference Details

Following [8, 9], stable outputs can be generated by reducing the temperature, which is a coefficient multiplied by the standard deviation of the prior. In this study, we generate stable outputs using a truncated normal distribution. The corresponding experiment is described in Section 6.2.

In existing TTS models [3, 5], the number of mel frames per text token is restricted to an integer because of hard attention. However, in KimchiSpeech, which uses Gaussian attention, this quantity can take a fractional value. Therefore, the speaking rate can be controlled more precisely, for example, by factors of 1.01× or 1.02×.

4.3. Model Configurations

We provide two model configurations: KimchiSpeech-W and KimchiSpeech-S. KimchiSpeech-W is a lightweight, weight-optimized version, whereas KimchiSpeech-S is a speed-optimized version designed for efficient GPU computation. For a small model size, it is advantageous to perform low-dimensional operations multiple times. Therefore, KimchiSpeech-W uses several blocks per layer with a small hidden dimension. On the other hand, to run efficiently on a GPU, KimchiSpeech-S uses only one block per layer with a large hidden dimension.

Table 1: *Model Configurations*

Hyperparameter	KimchiSpeech-W	KimchiSpeech-S
Embedding Dim	128	128
Text Encoder Dim	128	128
Number of Layers	4	5
Number of Blocks	4	1
Hidden Dim	128	512
z Dim	16	16

5. Experiments

LJSpeech-1.1 [24] was used as the training and test set for the experiments. A total of 523 samples from LJ001 and LJ002 were used as the test set, and 349 samples from LJ003 were used as the validation set. The MOS test was conducted with native English speakers through the Amazon Mechanical Turk service. Fifty samples were randomly selected from the test set. For the prior, as shown in Section 6.2, a truncated normal distribution with a range of $(-1, 1)$ was used. To convert the mel-spectrogram into a waveform, we used Parallel WaveGAN [25], as implemented by Kan Bayashi[2], with the pretrained model *ljspeech_parallel_wavegan.v3*. For Tacotron2, FastSpeech, and FastSpeech2, the pretrained models provided in ESPnet [26] were used. Speed measurements were recorded on an AMD Ryzen 7 2700X and an NVIDIA RTX 2080 Ti.

Table 2: *Model Comparison*

Model	MOS (95% CI)	RTF (GPU)	RTF (CPU)	Params
GT	4.28 ± 0.14	–	–	–
Tacotron2	3.29 ± 0.09	8.7×	2.2×	28.2M
FastSpeech	3.49 ± 0.05	187×	32×	23M
FastSpeech2	3.57 ± 0.07	205×	43×	27M
KimchiSpeech-W	$\mathbf{3.59 \pm 0.07}$	238×	**109**×	**3.3M**(31M)
KimchiSpeech-S	3.54 ± 0.08	**470**×	83×	18.6M(70M)

5.1. Audio Quality

As shown in Table 2, KimchiSpeech achieves quality comparable to or better than existing TTS models, reaching state-of-the-art performance.

5.2. Model Weight & Inference Speed

As shown in Table 2, the number outside the parentheses indicates the number of parameters required for inference. Specifically, this count includes the parameters of the text encoder and mel decoder, while excluding those of the STT model, the mel encoder, and the convolution blocks used for the posterior. The total number of model parameters required for training is shown in parentheses.

We measured the time from text input to mel-spectrogram output and compared it with the duration of the generated speech. Both KimchiSpeech variants are faster and smaller than existing models. KimchiSpeech-W has the smallest model size and is particularly optimized for CPU computation. Although KimchiSpeech-S has a larger model size than KimchiSpeech-W, it is optimized for GPU computation.

6. Ablation Study

The factors considered in determining the model are as follows.

- **1. Number of layers in the encoder and decoder**
- **2. Range of the truncated normal distribution**
- 3. Dimensions of the encoder and decoder
- 4. Configuration of the convolution block

Factors 3 and 4 can be determined by measuring the reconstruction loss at the beginning of training. However, Factors 1 and 2 were determined through listening tests after extended training.

[2] https://github.com/kan-bayashi/ParallelWaveGAN

6.1. Number of layers

Table 3 presents the MOS score and the reconstruction loss after 400k training steps. In KimchiSpeech-W, the MOS score was slightly higher with four layers than with five layers; however, the difference was not significant. In KimchiSpeech-S, the MOS score increased significantly when five layers were used. Because KimchiSpeech-S has only one convolution block per layer, long-range context can be captured only when a sufficient number of layers is provided. Meanwhile, the MOS score and the reconstruction loss were not consistently aligned.

Table 3: *Effect of the Number of Layers*

Model	Layers	MOS (95% CI)	Recon. Loss
KimchiSpeech-W	5	3.83 ± 0.13	**0.0424**
KimchiSpeech-W	4	$\mathbf{3.87 \pm 0.12}$	0.0431
KimchiSpeech-S	5	$\mathbf{3.87 \pm 0.12}$	0.0579
KimchiSpeech-S	4	3.53 ± 0.15	**0.0543**

6.2. Truncated Normal Distribution

Table 4 presents the truncation ranges required to produce high-quality speech. We compared outputs generated by sampling $\mathbf{z}$ from normal and truncated normal distributions. Consequently, outputs sampled from the normal distribution were unsatisfactory, and the best results were obtained with a truncated normal distribution over the range $(-1, 1)$.

Table 4: *Effect of Truncated Sampling Range*

Model	Range	MOS (95% CI)
KimchiSpeech-S	$(-1, 1)$	$\mathbf{3.90 \pm 0.09}$
KimchiSpeech-S	$(-2, 2)$	3.78 ± 0.10
KimchiSpeech-S	$(-3, 3)$	3.77 ± 0.09
KimchiSpeech-S	Not truncated	3.76 ± 0.10

7. Conclusion

We proposed KimchiSpeech, a TTS model that uses a hierarchical VAE and STT-aided attention. With the hierarchical VAE, high-quality mel-spectrograms with diverse prosody can be generated, while the STT-aided attention provides an attention alignment independently of the TTS model. In addition, the length of the generated speech can be flexibly controlled using Gaussian attention.

We also presented two configurations, KimchiSpeech-W and KimchiSpeech-S. The former is a lightweight, weight-optimized version, whereas the latter is a speed-optimized version. These two models are intended for different use cases depending on the application.

Compared with previously proposed state-of-the-art models, KimchiSpeech can generate outputs of comparable or superior quality. However, it remains difficult to faithfully reproduce the quality of real human speech. We plan to address these limitations in future work.

8. References

[1] Y. Wang, R. Skerry-Ryan, D. Stanton, Y. Wu, R. J. Weiss, N. Jaitly, Z. Yang, Y. Xiao, Z. Chen, S. Bengio *et al.*, "Tacotron: Towards end-to-end speech synthesis," *arXiv preprint arXiv:1703.10135*, 2017.

[2] J. Shen, R. Pang, R. J. Weiss, M. Schuster, N. Jaitly, Z. Yang, Z. Chen, Y. Zhang, Y. Wang, R. Skerrv-Ryan *et al.*, "Natural tts synthesis by conditioning wavenet on mel spectrogram predictions," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2018, pp. 4779–4783.

[3] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "Fastspeech: Fast, robust and controllable text to speech," in *Advances in Neural Information Processing Systems*, 2019, pp. 3165–3174.

[4] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in neural information processing systems*, 2014, pp. 2672–2680.

[5] Y. Ren, C. Hu, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "Fastspeech 2: Fast and high-quality end-to-end text-to-speech," *arXiv preprint arXiv:2006.04558*, 2020.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.

[7] C. Miao, S. Liang, M. Chen, J. Ma, S. Wang, and J. Xiao, "Flow-tts: A non-autoregressive network for text to speech based on flow," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7209–7213.

[8] J. Kim, S. Kim, J. Kong, and S. Yoon, "Glow-tts: A generative flow for text-to-speech via monotonic alignment search," *arXiv preprint arXiv:2005.11129*, 2020.

[9] Y. Lee, J. Shin, and K. Jung, "Bidirectional variational inference for non-autoregressive text-to-speech," in *International Conference on Learning Representations*, 2020.

[10] P. Liu, Y. Cao, S. Liu, N. Hu, G. Li, C. Weng, and D. Su, "Vara-tts: Non-autoregressive text-to-speech synthesis based on very deep vae with residual attention," *arXiv preprint arXiv:2102.06431*, 2021.

[11] M. Bińkowski, J. Donahue, S. Dieleman, A. Clark, E. Elsen, N. Casagrande, L. C. Cobo, and K. Simonyan, "High fidelity speech synthesis with adversarial networks," *arXiv preprint arXiv:1909.11646*, 2019.

[12] J. Donahue, S. Dieleman, M. Bińkowski, E. Elsen, and K. Simonyan, "End-to-end adversarial text-to-speech," *arXiv preprint arXiv:2006.03575*, 2020.

[13] R. Child, "Very deep vaes generalize autoregressive models and can outperform them on images," *arXiv preprint arXiv:2011.10650*, 2020.

[14] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[15] C. K. Sønderby, T. Raiko, L. Maaløe, S. K. Sønderby, and O. Winther, "Ladder variational autoencoders," *arXiv preprint arXiv:1602.02282*, 2016.

[16] A. Vahdat and J. Kautz, "Nvae: A deep hierarchical variational autoencoder," *arXiv preprint arXiv:2007.03898*, 2020.

[17] A. Graves, "Generating sequences with recurrent neural networks," *arXiv preprint arXiv:1308.0850*, 2013.

[18] E. Battenberg, R. Skerry-Ryan, S. Mariooryad, D. Stanton, D. Kao, M. Shannon, and T. Bagby, "Location-relative attention mechanisms for robust long-form speech synthesis," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 6194–6198.

[19] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International conference on machine learning*. PMLR, 2015, pp. 448–456.

[20] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.

[21] K. Park and J. Kim, "g2pe," https://github.com/Kyubyong/g2p, 2019.

[22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[23] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, "Cyclical annealing schedule: A simple approach to mitigating kl vanishing," *arXiv preprint arXiv:1903.10145*, 2019.

[24] K. Ito, "The lj speech dataset," https://keithito.com/LJ-Speech-Dataset/, 2017.

[25] R. Yamamoto, E. Song, and J.-M. Kim, "Parallel wavegan: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 6199–6203.

[26] T. Hayashi, R. Yamamoto, K. Inoue, T. Yoshimura, S. Watanabe, T. Toda, K. Takeda, Y. Zhang, and X. Tan, "Espnet-TTS: Unified, reproducible, and integratable open source end-to-end text-to-speech toolkit," in *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7654–7658.